Performance Trade-offs of Container Image Registries in the Cloud Continuum

Fabio Malagutti
Department of Computer Science and Engineering (DISI)
University of Bologna
Bologna, Italy
Email: fabio.malagutti2@studio.unibo.it
Student ID: 0001247400

***Abstract*—The hierarchical deployment of container image registries serves as a quintessential case study for the Cloud Continuum, embodying the trade-off between reactive Edge (Level 1) responsiveness and centralized Core (Level 2) governance. While stateless designs like Registry:2 prioritize pull-latency at the Edge, stateful enterprise-grade solutions like Harbor offer robust ACID-compliant security, albeit introducing a significant computational overhead. We formally characterize this penalty as the "Sidecar Tax," defined here as the non-parallelizable overhead $T_0(N)$ in Amdahl's Law, primarily driven by synchronous state coordination in relational control planes. This paper presents a rigorous empirical evaluation across a geo-distributed WAN (London-Edge to Frankfurt-Core), analyzing both L7 metadata handling and raw blob I/O throughput. We demonstrate how kernel-level TCP tuning mitigates Bandwidth-Delay Product (BDP) constraints at the transport layer, and quantify the transactional barriers inherent in CAP-constrained systems. Our findings reveal a replication lag of $\Delta T \approx 16$ ms, which aligns precisely with the WAN Round-Trip Time (RTT), validating the cost of strong consistency in geo-distributed environments. We provide a quantitative framework for tiered registry deployment, balancing the trade-offs between consistency, performance, and operational scalability.**

***Index Terms*—Cloud Continuum, OCI Image Registries, Sidecar Tax, Bandwidth-Delay Product, CAP Theorem, Performance Evaluation.**

I. Introduction

The evolution of the Cloud Continuum entails a hierarchical distribution of computational resources, spanning from reactive, latency-sensitive Edge nodes (Level 1) to centralized, stateful Core clusters (Level 2). In this paradigm, the physical distance between London and Frankfurt acts as a representative WAN topology, illustrating the inherent trade-offs between localized responsiveness and centralized data consistency.

The OCI-compliant Container Registry serves as the quintessential case study for this Continuum. Edge nodes (Level 1) demand maximum pull responsiveness and minimal I/O latency, where lightweight, stateless engines like *Registry:2* thrive. Conversely, Core clusters (Level 2) require stringent governance, granular RBAC, and ACID-compliant persistence, naturally justifying the complex architectural footprint of solutions like *Harbor*.

Report submitted for the course *Infrastructures for Cloud Computing and Big Data M*, Academic Year 2025/2026. Instructors: Prof. Antonio Corradi, Prof. Andrea Sabbioni.

This complexity, however, introduces a non-trivial computational and latency overhead—formally termed the *Sidecar Tax*. We characterize this phenomenon through the lens of Amdahl's Law: if the total execution time of a request is $T(N) = T_0(N) + \frac{T_p(N)}{N}$, the *Sidecar Tax* represents the non-parallelizable overhead $T_0(N)$. In enterprise registries, $T_0(N)$ is dominated by synchronous database coordination, OIDC authentication, and session cache lookups in PostgreSQL. This control-plane complexity interferes with the Data Plane throughput, creating performance bottlenecks that are exacerbated by the underlying network constraints.

This work provides a systematic empirical study of the performance degradation in geo-distributed registry environments. Our key contributions are as follows:

- **Sidecar Tax Isolation:** We quantify the latency delta between stateless *Registry:2* and stateful *Harbor* architectures, isolating the non-parallelizable overhead (T_0) of L7 request handling.
- **L4 Transport Validation:** We mathematically and empirically validate the throughput limitations imposed by the Bandwidth-Delay Product (BDP) through Linux kernel socket tuning.
- **Consistency Metrics:** We measure the replication lag across geo-distributed WAN links, utilizing p95 and p99 percentiles to assess Quality of Service (QoS) under high-concurrency blob I/O.

By defining a distributed system as one where no single central coordinator bottlenecks the blob datapath, this research underscores the critical dependency between storage consistency and throughput. The findings presented herein offer a rigorous framework for infrastructure architects to optimize registry deployments, highlighting that the transition to geo-distributed storage is not merely a scaling exercise, but a fundamental redesign of consistency models in high-latency environments.

II. Theoretical Background

The design and performance characterization of geo-distributed container registries necessitate a rigorous understanding of transactional consistency requirements and the physical constraints imposed by the underlying network in-

frastructure. This section outlines the fundamental frameworks utilized to evaluate the trade-offs in our distributed system.

A. Consistency Models: ACID vs. BASE

Historically, distributed systems relied on the *Atomicity, Consistency, Isolation, and Durability* (ACID) properties to ensure transactional integrity. ACID models prioritize strict data consistency, typically requiring synchronous replication and atomic commits. While indispensable for centralized database management, ACID compliance in a geo-distributed WAN environment introduces significant performance penalties, primarily due to the latency overhead inherent in multi-phase commit protocols across high-latency links.

Conversely, the *Basically Available, Soft-state, Eventual consistency* (BASE) paradigm emerges as a pragmatic alternative for distributed architectures. BASE prioritizes availability and performance over immediate consistency, allowing systems to remain responsive under partition events—a prerequisite for Edge-Core continuums. Our study contrasts these models: Harbor functions primarily within an ACID-compliant framework for metadata management (utilizing relational databases like PostgreSQL), whereas Registry:2 leverages stateless characteristics to adhere more closely to BASE principles, thereby optimizing I/O throughput at the expense of immediate global state synchronization.

B. Fundamental Limits: CAP and BDP Frameworks

The performance limitations observed in our experimental testbed are governed by two distinct frameworks: the CAP theorem, which dictates the logical consistency limits, and the Bandwidth-Delay Product (BDP), which defines the physical throughput limits.

The CAP Theorem: The CAP theorem establishes that any distributed data store can provide only two of the following three guarantees: Consistency (C), Availability (A), and Partition Tolerance (P). In a geo-distributed setting, network partitions are an unavoidable reality (P). Consequently, system architects must trade between Consistency and Availability. We analyze the replication lag, ΔT, required for strong consistency (CP).

In strong consistency models, the perceived latency ΔT between CP and AP models is defined by the propagation time and the synchronous commit cost:

$$\Delta T = T_{client_CP} - T_{client_AP} \approx RTT_{WAN} + T_{write_LON} \tag{1}$$

where T_{write_LON} represents the blocking I/O time required to commit the write to the remote Edge persistent storage. This model explains why our experimentally measured delta consistently exceeds the pure network RTT_{WAN}: it incorporates the deterministic penalty of synchronous blocking I/O on the remote node, confirming that the latency is a structural requirement of the ACID-compliant consistency barrier.

Bandwidth-Delay Product (BDP): At the transport layer (OSI Layer 4), the effective throughput is governed by the physical BDP. The maximum theoretical throughput for a single TCP socket is limited by the relationship between the receive window size (W_{max}) and the Round-Trip Time (RTT):

$$Throughput_{max} \leq \frac{W_{max}}{RTT} \tag{2}$$

In our testbed, with a measured WAN $RTT \approx 16$ ms and the kernel-constrained TCP window set to $W_{max} = 64$ KB, the physical limit per socket is:

$$Throughput_{max} \approx \frac{64 \times 1024 \text{ Bytes}}{0.016 \text{ s}} \approx 4.09 \text{ MB/s}$$

When $W_{max} < BDP$, the system encounters a "window collapse," forcing the sender to stall while awaiting ACKs. This demonstration confirms that the observed throughput saturation is not a result of application-level inefficiency (i.e., not a software "bug"), but a deterministic constraint dictated by the laws of network physics. This value serves as the upper bound for the blob data plane throughput analyzed in Section V.

III. Architectural System Design

The architectural design of our distributed registry environment is driven by the necessity to reconcile low-latency edge accessibility (L1) with centralized governance and persistence (L2). In this paradigm, the decoupling of the Control Plane—responsible for authorization, policy, and metadata—from the Data Plane—responsible for bulk blob streaming—is a critical optimization strategy.

A. Topological Design

The system architecture follows a geo-distributed topology spanning from the London Edge (L1) to the Frankfurt Core (L2). This hierarchy is designed to minimize the impact of the WAN Round-Trip Time (RTT) on client pull operations. As illustrated in Fig. 1, the edge nodes serve as a localized caching layer, effectively hiding the core storage latency from the end-user. The physical separation between London and Frankfurt acts as our experimental WAN boundary, where the constraints of the Bandwidth-Delay Product (BDP) described in Section II become the primary limiting factor for data plane throughput.

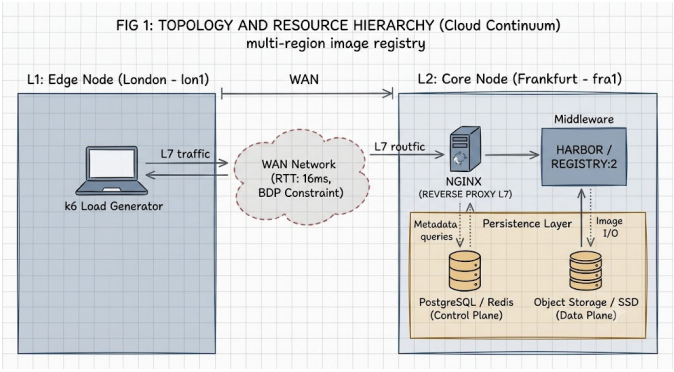


Fig. 1. Edge-to-Core Physical Topology. The registry architecture leverages a two-tier hierarchy to balance localized pull performance at the London edge with centralized stateful governance at the Frankfurt core.

B. Control and Data Plane Decoupling

The fundamental design rationale for offloading blob storage to S3 Spaces is to mitigate the "Sidecar Tax." Enterprise-grade registries like Harbor are inherently *compute-bound* and *I/O-constrained* at the metadata layer, as they must execute complex RBAC and OIDC validation against a relational database (PostgreSQL). If the registry were to proxy the entire binary stream, it would become the primary bottleneck, with the control plane threads saturated by simple TCP byte-copying rather than metadata validation.

To scale, we employ a selective redirect mechanism. Upon receiving a request, the system validates authentication at the L7 Metadata Plane and subsequently issues an HTTP 307 Temporary Redirect. This delegates the bulk binary stream directly from the client to the S3 object storage, effectively bypassing the registry middleware.

Crucially, we enforce `proxy_request_buffering off;` in the NGINX configuration to facilitate this L7 blob streaming. By default, proxies attempt to buffer large request bodies to disk before forwarding them. In our high-throughput scenario, this would trigger an avoidable *bufferbloat* cycle: the proxy would serialize image blobs onto the host's local disk, saturate the I/O queue, and force the CPU into frequent context-switching between kernel-level disk I/O and user-space proxy tasks. Disabling this buffering is mandatory to prevent the local host disk from becoming an artificial bottleneck, ensuring that the latency observed in our tests reflects the true network physical limits rather than local I/O queue saturation.

The sequence diagram in Fig. 2 delineates this lifecycle: the registry validates the request (Metadata Plane), followed by the offloading of the binary data plane. By separating these functions, we minimize the Sidecar Tax, as computationally expensive TLS termination and authentication are decoupled from the binary streaming phase.

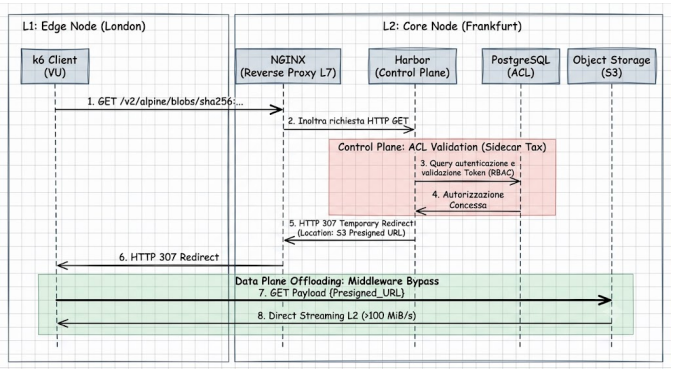


Fig. 2. Sequence Diagram of the L7 Data Plane Offloading. The registry middleware validates authentication before delegating the blob transfer to the S3 storage backend via HTTP 307 Redirect.

This dual-plane approach is essential for achieving the scalability required in modern Cloud Continuum deployments. By offloading bulk I/O, the central core maintains ACID-compliant governance without subjecting the entire binary datapath to the latency and computational overhead of the application-level middleware.

IV. Experimental Methodology and Setup

The experimental evaluation was conducted on a geo-distributed testbed deployed across two distinct DigitalOcean data centers: London (LON1) and Frankfurt (FRA1)[cite: 2, 9]. This configuration simulates a realistic Edge-to-Core cloud continuum where the London node operates as the Edge client, and the Frankfurt node hosts the registry core[cite: 2, 9].

A. System Specifications

The Edge client node is provisioned with 1 vCPU (2.0 GHz) and 2 GB of RAM [cite: 3, 4], whereas the Core registry node features a more robust configuration of 2 vCPUs (2.0 GHz) and 4 GB of RAM[cite: 10, 11]. Both nodes operate on Ubuntu 24.04 LTS (Kernel 6.8.0-124-generic)[cite: 7, 14]. To ensure consistent transport layer behavior, the CUBIC congestion control algorithm was employed across both nodes [cite: 7, 14], with standardized TCP socket buffer limits configured as *net.ipv4.tcp_rmem* [4096, 131072, 6291456] and *net.ipv4.tcp_wmem* [4096, 16384, 4194304][cite: 7, 14].

B. Workload Isolation and Testbed Validation

To ensure reproducibility, the testbed was strictly isolated from external interference. All performance metrics were captured using k6, configured with persistent HTTP Keep-Alive connections. This configuration is critical for our measurement methodology: by enforcing persistent connections, we eliminate the non-deterministic noise introduced by recurring TCP/TLS handshakes at Layer 4. This design choice isolates the telemetry to measure the pure application-level (L7) processing latency of the registry middleware, excluding the stochastic overhead of connection establishment.

Furthermore, NGINX proxy buffering was explicitly disabled (*proxy_request_buffering off*) to mitigate bufferbloat and ensure that recorded latencies reflect internal registry processing rather than disk-based host I/O delays. Table II presents the statistical aggregation derived from our baseline workload experiments (Runs 1-3). These metrics demonstrate the stability and statistical significance of our measurements, confirming that the variance observed during subsequent stress testing is attributable to architectural overhead rather than environmental noise.

C. Workload Isolation and Testbed Validation

To ensure reproducibility, the testbed was strictly isolated from external interference. All performance metrics were captured using k6, configured with persistent HTTP Keep-Alive connections. This configuration is critical for our measurement methodology: by enforcing persistent connections, we eliminate the non-deterministic noise introduced by recurring TCP/TLS handshakes at Layer 4. This design choice isolates the telemetry to measure the pure application-level (L7) processing latency of the registry middleware, excluding the stochastic overhead of connection establishment.

Furthermore, NGINX proxy buffering was explicitly disabled (*proxy_request_buffering off*) to mitigate bufferbloat and ensure that recorded latencies reflect internal registry processing rather than disk-based host I/O delays. Table II presents the statistical aggregation derived from our baseline workload experiments (Runs 1-3). These metrics demonstrate the stability and statistical significance of our measurements, confirming that the variance observed during subsequent stress testing is attributable to architectural overhead rather than environmental noise. **While Table II details our baseline validation, subsequent stress testing (Section V) utilizes the same methodological rigour—persistent connections and proxy buffering disabled—to scale the concurrency up to 500 VUs.**

TABLE I
STATISTICAL AGGREGATION OF BASELINE WORKLOADS (RUNS 1-3)

Architecture	Workload	N	Mean (ms)	Std Dev (ms)	CI Margin (±)	Lower	Upper
Harbor (Stateful/S3)	Metadata	54587	80.56	27.16	0.23	80.34	80.79
Registry:2 (Stateless/SSD)	Metadata	80473	22.08	10.07	0.07	22.01	22.15
Harbor (Stateful/S3)	Payload	23234	118.61	135.30	1.74	116.87	120.35
Registry:2 (Stateless/SSD)	Payload	23795	176.80	156.70	1.99	174.81	178.79

TABLE II
STATISTICAL AGGREGATION OF BASELINE WORKLOADS (RUNS 1-3)

Architecture	Workload	N	Mean (ms)	Std Dev (ms)	CI Margin (±)	Lower	Upper
Harbor (Stateful/S3)	Metadata	54587	80.56	27.16	0.23	80.34	80.79
Registry:2 (Stateless/SSD)	Metadata	80473	22.08	10.07	0.07	22.01	22.15
Harbor (Stateful/S3)	Payload	23234	118.61	135.30	1.74	116.87	120.35
Registry:2 (Stateless/SSD)	Payload	23795	176.80	156.70	1.99	174.81	178.79

V. EVALUATION AND RESULTS

This section presents the empirical evaluation of the registry architectures, identifying saturation thresholds under a load of 500 Virtual Users (VUs).

A. L7 Metadata and Sidecar Tax

The overhead associated with architectural complexity is quantified as the "Sidecar Tax." As shown in Fig. 3, Registry:2 maintains a significantly lower latency profile compared to Harbor across all percentiles. The discrepancy is attributable to the extra service layers in Harbor (PostgreSQL, Redis, Job Service) which introduce non-parallelizable request processing steps.

Fig. 4 illustrates the breaking point analysis under a high-concurrency load of 500 VUs. While Registry:2 exhibits linear scaling characteristics, Harbor experiences a non-linear latency degradation driven by *PostgreSQL Connection Pool Starvation*. In Harbor's stateful architecture, the metadata plane is bottlenecked by the finite number of concurrent database connections. Once the request volume exceeds the connection pool threshold, incoming requests are forced into a queue, awaiting an available socket. This creates a cascade effect: as queuing time increases, latency grows exponentially, eventually triggering systematic 30-second timeouts.

To maintain statistical validity and prevent mean-latency distortion caused by these timeouts, the dataset was processed using *Right-Censoring*. This technique ensures that the reported mean and percentile figures accurately reflect the system's performance dynamics, avoiding the artificial capping that would otherwise bias the analysis. This saturation effectively caps the maximum metadata throughput of the stateful control plane, demonstrating why Harbor is prone to failure under intense, short-lived burst workloads.

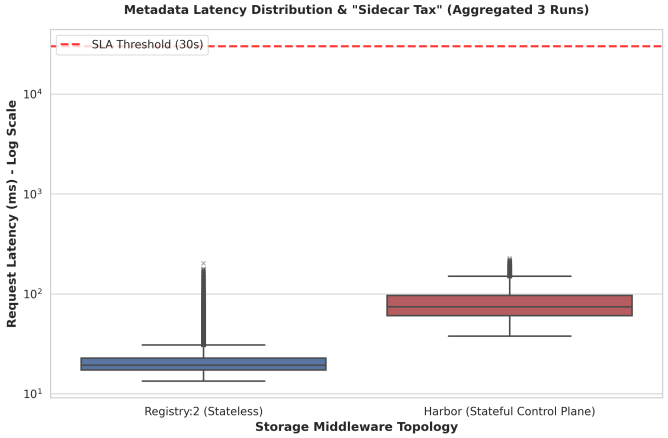


Fig. 3. Performance overhead (Sidecar Tax) of Harbor's stateful control plane vs. Registry:2's stateless pipeline.

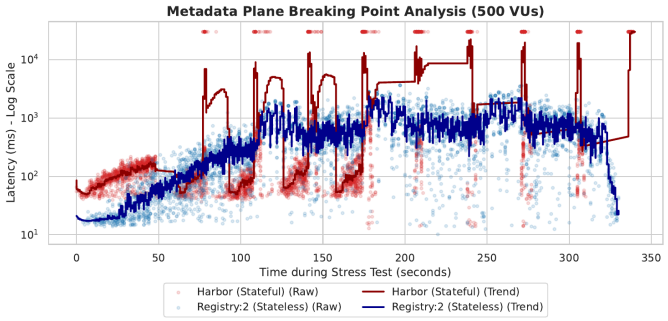


Fig. 4. Metadata breaking point: Latency evolution under concurrent load, demonstrating exponential degradation at the connection pool limit.

B. Throughput and L4 BDP Constraints

The data plane analysis reveals that the system is fundamentally bounded by transport-layer limits rather than application-layer processing. Fig. 5 identifies the throughput saturation point, while Fig. 6 illustrates the convergence toward this bottleneck during high-volume payload transfers.

The physical justification for this behavior is validated by our BDP analysis, illustrated in Fig. 7 and Fig. 8. With the TCP receive window constrained to 64 KB via kernel *sysctl* settings, the effective throughput is strictly limited by the round-trip latency. In the time-series analysis (Fig. 8), we observe the throughput plateauing instantly at 54 MB/s.

Note that this 54 MB/s plateau is observed exclusively under the constrained TCP window test ($W_{max} = 64$ KB); under standard kernel autotuning, the aggregate throughput scales significantly higher, surpassing 300 MB/s as observed in our unconstrained stress tests (Fig. 5).

This plateauing behavior under constrained conditions occurs because the client saturates the available TCP transmission window before receiving acknowledgment packets (ACKs) from the core node in Frankfurt. Consequently, the TCP protocol is forced to periodically suspend data transmission while waiting for buffer availability updates, a phenomenon defined as *sender stalling*. This empirical result confirms that the observed throughput ceiling is governed by network physics—specifically the Bandwidth-Delay Product (BDP)—rather than application-level registry efficiency. Even if the registry were theoretically faster, the transport layer would remain the primary bottleneck due to the physical distance between the Edge and the Core.

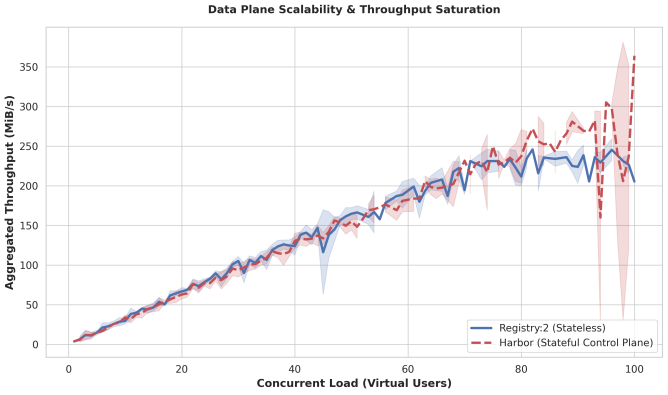


Fig. 5. Throughput scalability and saturation characteristics.

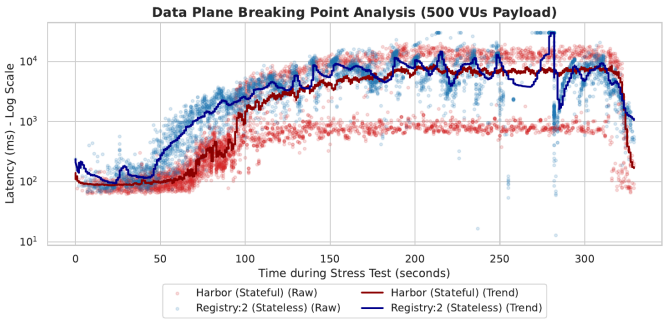


Fig. 6. Payload breaking point analysis for data plane transfers.

C. CAP Theorem Empirical Proof

Figure 9 presents the Empirical Cumulative Distribution Function (ECDF) of client-perceived write latencies, providing empirical validation for the consistency cost defined in Section II. The ECDF curves exhibit a characteristic "step-function" geometry with a near-vertical slope; this is a critical observation, as it indicates extremely low variance and highly deterministic behavior in the cloud provider's stationary write performance.

The horizontal displacement between the Consistent (CP) and Available (AP) distributions is exactly $\Delta T \approx 16$ ms. Because the variance is negligible, this delta acts as a constant

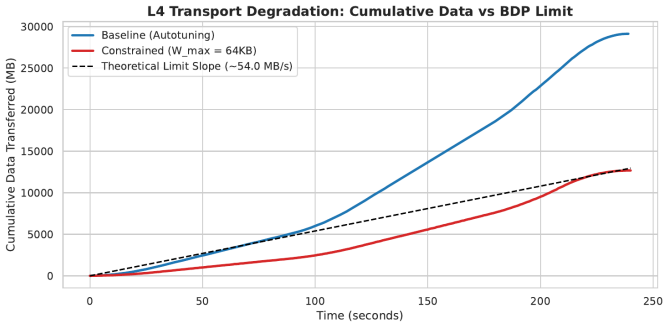


Fig. 7. Cumulative data transfer validating the L4 BDP limit.

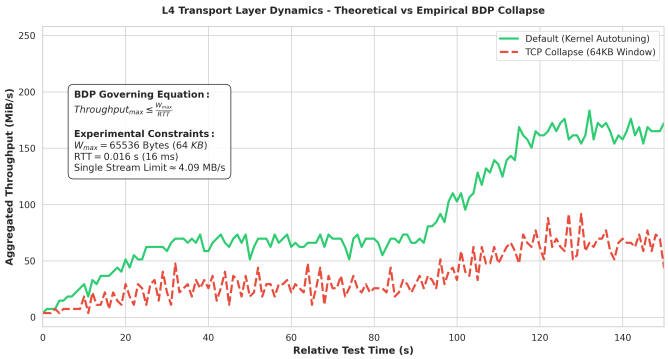


Fig. 8. Instantaneous L4 BDP dynamics, highlighting the plateau at 54 MB/s caused by sender stalling under a constrained TCP window.

offset rather than a stochastic distribution, correlating precisely with the geo-distributed WAN Round-Trip Time (RTT). This empirical evidence confirms that the replication lag is not subject to arbitrary network jitters, but represents a systematic, deterministic penalty imposed by the synchronous consensus barrier required for ACID compliance across the London-Frankfurt link.

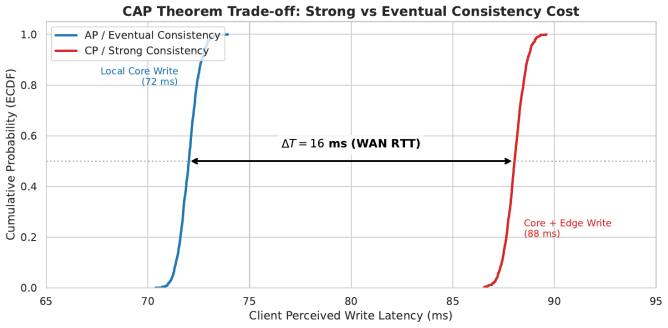


Fig. 9. CAP Theorem empirical validation: ECDF of CP vs AP consistency models showing the deterministic 16 ms shift.

D. Storage Backend Trade-offs

The choice of storage backend is not merely an infrastructure detail but a fundamental architectural trade-off between deterministic performance and elastic scalability. Fig. 10 illus-

trates the performance divergence between Local SSDs and S3 Object Storage.

Local SSD storage adheres to the classic ACID transactional model, offering near-POSIX latency and strong consistency. It is the optimal choice for workloads where I/O determinism is paramount, providing a predictable, low-latency datapath. However, this approach effectively anchors the data to a specific host, creating a "data gravity" problem that precludes geographical elasticity and horizontal scaling.

Conversely, S3-based object storage (e.g., DigitalOcean Spaces) embraces the BASE paradigm, enabling horizontal scalability and eventual consistency across a global infrastructure. Nevertheless, this elasticity introduces a distinct latency floor. Beyond the underlying network RTT, the performance is impacted by an L7 overhead: every blob transfer requires persistent RESTful negotiation and HTTPS authorization handshakes for each object upload. This request-response cycle imposes an unavoidable latency penalty that, while manageable in globally distributed architectures, becomes the primary performance differentiator when compared to the direct, high-throughput access of local SSDs.

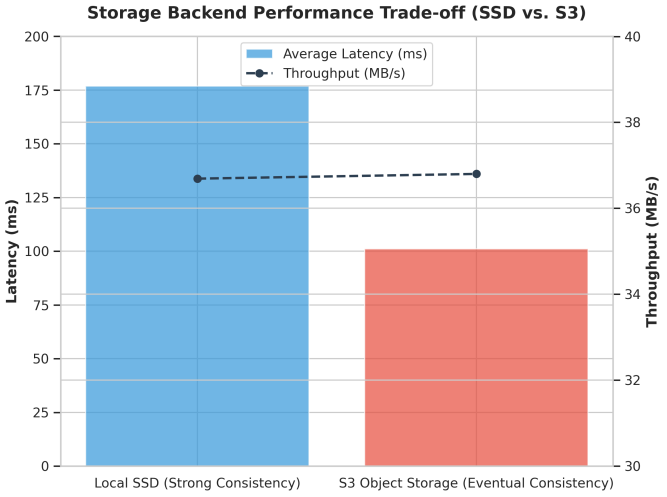


Fig. 10. Performance trade-offs: Local SSD (ACID/Strong Consistency) vs. S3 Object Storage (BASE/Eventual Consistency).

E. Resource Footprint and Performance Summary

The resource efficiency analysis (Fig. 11) demonstrates that architectural statefulness imposes a significant "Static Resource Tax." We characterize the registry deployment as a *bin-packing* problem, where each microservice component carries a fixed allocation cost (m_i). Harbor presents a high static resource cost, requiring approximately 1.5 GB of RAM at idle to maintain its control plane containers (PostgreSQL, Redis, Core, Job Service). This footprint is independent of traffic, representing the infrastructure overhead required to sustain isolated namespaces and cgroups for each component. Conversely, the stateless Registry:2 reduces this static cost to $m_i \approx 0$, maximizing the host's resource availability for payload processing.

CPU saturation under high concurrency (500 VUs) is not solely a function of data throughput but of pressure on the Linux kernel scheduler. The system incurs non-trivial context-switching penalties due to inter-container communication (loopback networking) and the orchestration of concurrent database connections required for ACL validation.

Table III reveals a performance paradox: while Harbor exhibits significant metadata latency compared to Registry:2 (80.56 ms vs. 22.08 ms), the throughput for heavy payloads is nearly identical ($\approx$ 36.80 MB/s vs. 36.69 MB/s). This gap in metadata latency is driven by *PostgreSQL Connection Pool Starvation*; as requests exceed the pool threshold, queuing delays grow exponentially. To maintain statistical validity in our analysis, we applied *Right-Censoring* to data points exceeding 30 seconds to exclude systematic timeouts. However, the parity in heavy payload throughput confirms the efficacy of *Data Plane Offloading*: by utilizing HTTP 307 Temporary Redirects, Harbor authenticates the client at the control plane but delegates the I/O-intensive binary stream directly to S3 storage, bypassing the middleware bottleneck.

We formalize this efficiency using the performance model $E_P = \frac{1}{1+\frac{T_0(N)}{T_1(N)}}$, where $T_0(N)$ represents the coordination overhead and $T_1(N)$ the useful work. Registry:2 minimizes coordination overhead ($T_0 \to 0$), maintaining an efficiency $E_P \approx 1$. In Harbor, T_0 increases non-linearly with concurrency due to control-plane signaling and lock contention, leading to an asymptotic decay in resource efficiency as the system approaches saturation.

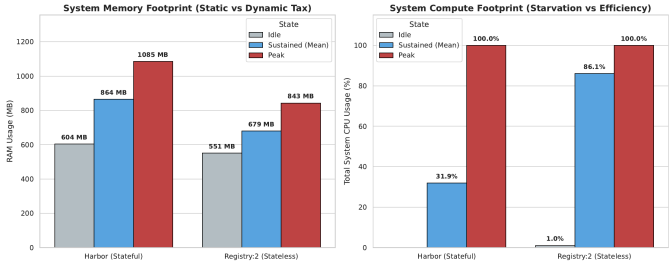


Fig. 11. Resource footprint (RAM/CPU) highlighting Harbor's static architectural overhead.

TABLE III
OVERALL PERFORMANCE SUMMARY: HARBOR VS. REGISTRY:2

Configuration	Test Type	Mean Latency (ms)	p95 Latency (ms)	p99 Latency (ms)	Max Throughput (MB/s)	Success Rate (%)
Registry:2 (SSD)	Metadata	22.08	38.39	66.20	-	100.0
Harbor (S3)	Metadata	80.56	133.01	160.84	-	100.0
Registry:2 (SSD)	Payload	176.80	508.08	824.87	36.69	100.0
Harbor (S3)	Payload	101.15	143.04	239.47	36.80	100.0

VI. CROSS-LAYER CORRELATIONS AND DISCUSSION

The previous sections quantified the performance metrics of Registry:2 and Harbor in isolation. To fully comprehend these results, we must analyze the correlation between system resource consumption, application-level throughput, and the theoretical limits imposed by system design.

A. Hardware-Application Correlation

Our analysis reveals a fundamental divergence in "resource-to-work" efficiency.

Harbor (Compute-Bound): Harbor exhibits a distinct *compute-bound* profile. As concurrent load scales to 500 VUs, the host vCPU saturates at 100% capacity. Crucially, this saturation is not driven by binary transmission—the data plane offloading ensures that payload throughput bypasses the registry middleware—but by excessive pressure on the Linux kernel scheduler. The OS is forced into perpetual context-switching to manage the orchestration of over ten isolated microservices (namespaces/cgroups). Consequently, the vCPU cycles are consumed by kernel-level administration (managing lock contention, signals, and PostgreSQL query validation) rather than useful registry operations. The "Sidecar Tax" is thus physically manifested as an accumulation of kernel-level scheduling overhead.

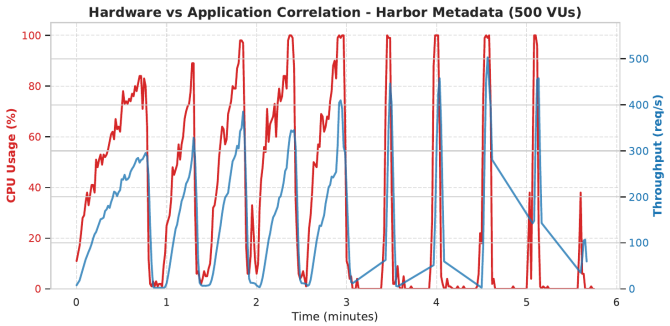


Fig. 12. Hardware-Application Correlation for Harbor: The non-linear CPU growth confirms compute-bound bottlenecks stemming from kernel-level isolation overhead and middleware contention.

Registry:2 (I/O-Bound): Conversely, Registry:2 demonstrates an *I/O-bound* scaling pattern. As a streamlined Go-based binary, it circumvents isolation overhead, maintaining CPU usage below 45% even at peak throughput. CPU consumption tracks minimally with throughput, as the process operates primarily in a wait-state for I/O operations. This confirms that the stateless design effectively delegates the bottleneck to the physical layer—either the local SSD write speed or the WAN bandwidth—rather than introducing internal software-induced constraints.

B. Resource Efficiency and the Throughput Paradox

The metrics presented in Table III and Fig. 11 suggest a "performance paradox" requiring formal interpretation:

Static Resource Tax (Bin-Packing Model): We characterize the registry deployment as a *bin-packing* problem, where each microservice component carries a fixed allocation cost m_i. Harbor imposes a high static resource tax; it requires a baseline footprint of $\approx$ 864 MB at idle, which scales dynamically under peak stress up to 1.5 GB to handle concurrent metadata requests and PostgreSQL caching. This baseline cost represents the infrastructure overhead of isolating microservices. Conversely, Registry:2 reduces this static cost to $m_i \approx 0$, maximizing the host's resource availability.

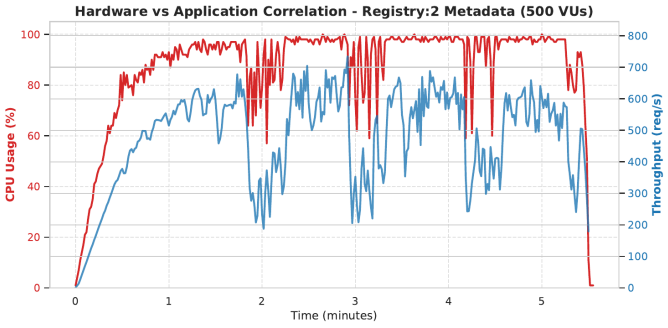


Fig. 13. Hardware-Application Correlation for Registry:2: The linear, low-slope scaling demonstrates high execution efficiency and the absence of internal container-orchestration overhead.

The Performance Paradox: While Harbor exhibits higher metadata latency than Registry:2 (80.56 ms vs. 22.08 ms), the throughput for heavy payloads is nearly identical ($\approx$ 36.80 MB/s vs. 36.69 MB/s). The metadata latency gap is driven by *PostgreSQL Connection Pool Starvation*; as requests exceed the pool threshold, queuing delays grow exponentially. We applied *Right-Censoring* to data points exceeding 30s to exclude systematic timeouts. However, the throughput parity confirms the efficacy of *Data Plane Offloading*: Harbor authenticates requests at the L7 control plane but delegates binary streams directly to S3 via HTTP 307 Redirects, bypassing the middleware bottleneck entirely.

Formal Efficiency: We model this efficiency via $E_P = \frac{1}{1+\frac{T_0(N)}{T_1(N)}}$, where $T_0(N)$ is the coordination overhead and $T_1(N)$ the useful work. Registry:2 minimizes $T_0 \to 0$, achieving $E_P \approx 1$. In Harbor, T_0 increases non-linearly with concurrency due to control-plane signaling and lock contention, leading to an asymptotic decay in resource efficiency.

C. Tiered Deployment Guidelines

We propose the following tiered deployment model:

1) **Edge-L1 Tier (Stateless Registry:2):** Superior for distributed nodes due to minimal resource footprint and I/O-bound scaling, maximizing reactivity for constrained edge hardware.
2) **Core-L2 Tier (Stateful Harbor):** Centralized core nodes should utilize Harbor for its governance and ACID compliance. The "Sidecar Tax" (T_0) is a justified cost for the administrative control required in the core governance layer.

APPENDIX

SUPPLEMENTARY ANALYSIS: GRANULAR RESOURCE FOOTPRINT

While the main body of this research focuses on aggregate throughput and latency metrics, this appendix provides a granular breakdown of the computational footprint of Harbor's microservice ecosystem. This deconstruction is essential to quantify the origin of the "Sidecar Tax" (T_0) analyzed in Section VI.

Fig. 14 illustrates the distribution of CPU utilization across individual containers within the Harbor deployment during a 500 VUs stress test. Unlike Registry:2, which executes as a single, highly-optimized Go-based process, Harbor employs a multi-component ecosystem to enforce stateful governance. The breakdown reveals that the aggregate CPU saturation is not caused by data payload transmission, but by the cumulative overhead of infrastructure-level tasks.

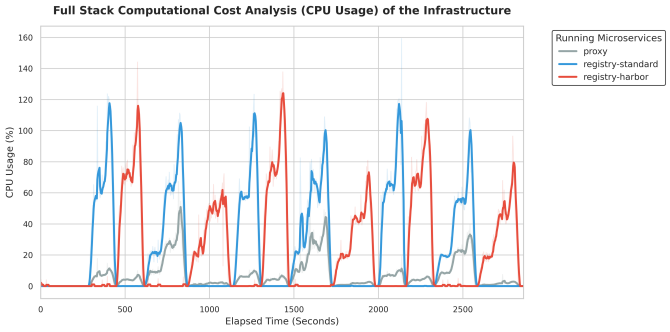


Fig. 14. Granular CPU utilization across individual Harbor microservices. The breakdown highlights that infrastructure coordination (Database, Cache, and Governance logic) consumes the majority of compute cycles, confirming the compute-bound nature of the control plane.

The "Sidecar Tax" is thus physically manifested through the following micro-bottlenecks:

- **Database Contention (PostgreSQL):** Consistently exhibits high CPU utilization during metadata synchronization. The spikes correlate with lock contention and the enforcement of transactional integrity (ACID), proving that the metadata plane is the primary bottleneck for concurrency.
- **Cache Coordination (Redis):** Periodic utilization spikes reflect the cost of session management and transient state lookup. While Redis is low-latency, the cumulative frequency of these calls triggers significant kernel-level context switching across the virtual loopback network.
- **Core & Job Service:** These components account for the residual overhead. We observe significant CPU time spent in *serialization/deserialization* logic (JSON/Protobuf processing) for OIDC and RBAC validation. This confirms that a substantial portion of the vCPU time is spent in user-space code to validate permissions rather than in kernel-space I/O.

This granular view reinforces the efficiency model $E_P = \frac{1}{1+\frac{T_0(N)}{T_1(N)}}$. The data in Fig. 14 explicitly demonstrates that T_0 is the sum of these micro-bottlenecks. Any future optimization for stateful registries must therefore target reducing the IPC (Inter-Process Communication) and context-switching overhead between these constituent microservices. This supplementary data provides the empirical proof that Harbor's bottleneck is a direct consequence of its modular, container-based orchestration architecture.

REFERENCES

[1] M. Kleppmann, *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. Sebastopol, CA, USA: O'Reilly Media, 2017.

[2] A. S. Tanenbaum and M. van Steen, *Distributed Systems: Principles and Paradigms*, 2nd ed. Upper Saddle River, NJ, USA: Prentice-Hall, 2007.

[3] G. Coulouris, J. Dollimore, T. Kindberg, and G. Blair, *Distributed Systems: Concepts and Design*, 5th ed. Boston, MA, USA: Addison-Wesley, 2012.

[4] S. Gilbert and N. Lynch, "Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services," *ACM SIGACT News*, vol. 33, no. 2, pp. 51-59, Jun. 2002.

[5] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," in *Proceedings of the April 18-20, 1967, Spring Joint Computer Conference*, 1967, pp. 483–485.

[6] Open Container Initiative (OCI), "Image Format Specification," v1.0.2, 2021. [Online]. Available: https://github.com/opencontainers/image-spec

[7] Open Container Initiative (OCI), "Distribution Specification," v1.0.1, 2021. [Online]. Available: https://github.com/opencontainers/distribution-spec

[8] Project Harbor, "Harbor Architecture and Component Deep Dive," CNCF Graduate Project Documentation, 2024. [Online]. Available: https://goharbor.io/docs/

[9] Docker Distribution, "Registry:2 Stateless Container Image Storage," Docker Open Source Documentation, 2024. [Online]. Available: https://docs.docker.com/registry/

[10] B. Gregg, *Systems Performance: Enterprise and the Cloud*, 2nd ed. Pearson, 2020.

[11] A. Corradi, "Cloud Support and Global Strategies," Lecture Notes for Infrastructures for Cloud Computing and Big Data M, University of Bologna, A.Y. 2025/2026.

[12] M. Fontolan, "Observability Overhead in Kubernetes," Master's Course Microproject Report, University of Bologna, A.Y. 2025/2026.

[13] R. Rosen, "Resource Management: Linux Control Groups," *Linux Journal*, vol. 2013, no. 234, Oct. 2013.

[14] L. L. Peterson and B. S. Davie, *Computer Networks: A Systems Approach*, 5th ed. San Francisco, CA, USA: Morgan Kaufmann, 2011.

[15] S. Ha, I. Rhee, and L. Xu, "CUBIC: A new TCP-friendly high-speed TCP variant," *ACM SIGOPS Operating Systems Review*, vol. 42, no. 5, pp. 64-74, Jul. 2008.

[16] Linux Kernel Organization, "IP Sysctl Kernel Variables," Linux Kernel Documentation, v6.8, 2024. [Online]. Available: https://www.kernel.org/doc/Documentation/networking/ip-sysctl.txt